

Laboratorio di Programmazione Web
(laurea in Scienze e Tecnologie Multimediali)

A.A. 2026/2027

Web link alle pagine del corso:

http://elearning.uniud.it/moodle/course/view.php?id=6288

Server side programming
PHP scripting language

I materiali in queste slide sono tratti in parte da:

V. Della Mea, L. Di Gaspero, I. Scagnetto,"Programmazione lato server", Apogeo.
http://www.w3.org/
http://en.wikipedia.org (Wikipedia)
http://www.php.net

The PHP language syntax

Operators and functions

The PHP language syntax

I. Operators

- Operators are language constructs that allow to process data. They take one or more values as input and returns a value as output.

- In PHP operators can be classified in:
  - basic operators
    - Arithmetic, Increment/decrement, Logical
    - Assignment and comparison operators
    - Operators on strings
    - Operators on arrays
  - type cast (conversion) operators
  - PHP specific operators.

The PHP language syntax

I. Basic operators

- Arithmetic operators: *, /, +, -, %
- The division operator ("/") returns a float value unless the two operands are integers (or strings that get converted to integers) and the numbers are evenly divisible, in which case an integer value will be returned.
- Operands of modulus (“%”) are converted to integers (by truncation) before processing.

```
1  <?php
2   echo (1 % 3);            // prints 1
3   echo (2 % 3);            // prints 2
4   echo (3 % 3);            // prints 0
5   echo (4 % 3);            // prints 1
6   echo (5 % 3);            // prints 2
7   echo (-5 % 3);           // prints -2
8   echo (6 % 3);          // prints 0
9   ...
10 ?>
```

The PHP language syntax

I. Basic operators

- Assignment operators (=):

```php
<?php
 $a =  5; // a has been assigned the value 5 .
 $a = ($b = 4) + 5; // a is equal to 9 now, and b has been set to 4.
?>
```

- Combined operators (+=,-=, …):

```php
<?php
 $a = 3;
 $a += 5; // sets variable to 8
?>
```

The PHP language syntax

I. Basic operators

- Comparison operators

Example	Name	Result
$a == $b	Equal	TRUE if $a is equal to $b
$a === $b	Identical	TRUE if *$a* is equal to *$b*, and they are of the same type.
$a != $b	Not equal	TRUE if *$a* is not equal to *$b* after type juggling.
$a <> $b	Not equal	TRUE if *$a* is not equal to *$b* after type juggling.
$a !== $b	Not identical	TRUE if *$a* is not equal to *$b*, or they are not of the same type.
$a < $b	Less than	TRUE if $*a* is strictly less than *$b*.
$a > $b	Greater than	TRUE if *$a* is strictly greater than *$b*.
$a <= $b	Less than or equal to	TRUE if $*a* is less than or equal to *$b*.
$a >= $b	Greater than or equal to	TRUE if $*a* is greater than or equal to *$b*.

- N.B.: if the comparison involves numerical strings, then each string is converted to a number and the comparison is performed numerically

The PHP language syntax

I. Basic operators

- String Concatenation (“.” and “.=”):

```php
<?php
 $a = "Hello ";
 $b = $a . "World!"; // now $b contains "Hello World!"

 $a = "Hello ";
 $a .= "World!";     // now $a contains "Hello World!"
?>
```

The PHP language syntax

I. Basic operators

- Incrementing/decrementing operators (++$a, $a++, --$a, $a--)

```php
<?php
  echo "<h3>Postincrement</h3>";
  $a = 5;
  echo "Should be 5: " . $a++ . "<br />\n";
  echo "Should be 6: " . $a . "<br />\n";

 echo "<h3>Preincrement</h3>";
 $a = 5;
 echo "Should be 6: " . ++$a . "<br />\n";
 echo "Should be 6: " . $a . "<br />\n";
?>
```

The PHP language syntax

I. Basic operators

- Logical operators: operations on booleans

Example	**Name**	**Result**
$a and $b	And	**TRUE** if both *$a* and *$b* are **TRUE**.
$a or $b	Or	**TRUE** if either *$a* or *$b* is **TRUE**.
$a xor $b	Xor	**TRUE** if either *$a* or *$b* is **TRUE**, but not both.
! $a	Not	**TRUE** if *$a* is not **TRUE**.
$a && $b	And	**TRUE** if both *$a* and *$b* are **TRUE**.
$a || $b	Or	**TRUE** if either *$a* or *$b* is **TRUE**.

The PHP language syntax

II. Type conversion operators (cast and juggling)

- Cast operators allows explicit conversion of variable type:

  - (int) or (integer) converts the value into an integer;
  - (bool) or (boolean) converts the value into a boolean;
  - (float) or (double) or (real) converts the value into a float;
  - (string) converts the value into a string;
  - (array) converts the value into an array;

- E.g.:

```php
<?php
 $foo = 10;    // $foo is an integer
 $bar = (boolean) $foo;    // $bar is a boolean
?>
```

```php
<?php
  $foo = 10;              // $foo is an integer
  $str = "$foo";          // $str is a string
  $fst = (string) $foo; // $fst is also a string
?>
```

The PHP language syntax

II. Type conversion operators (cast and juggling)

- Type juggling refers to *automatic* type conversion

- E.g., PHP automatically convert '+‘ operands as follows:

  If either operand is a float, then both operands are evaluated as floats, and the result will be a float.
  Otherwise, the operands will be interpreted as integers, and the result will also be an integer.

- E.g.:

```php
<?php
 $foo = "0";  // $foo is string (ASCII 48)
 $foo += 2;   // $foo is now an integer (2)
 $foo = $foo + 1.3;  // $foo is now a float (3.3)
 $foo = 5 + "10 Little Piggies"; // $foo is integer (15)
 $foo = 5 + "10 Small Pigs";     // $foo is integer (15)
?>
```

The PHP language syntax

II. Type conversion operators (cast and juggling)

- Type access functions: *is_boolean()*, *is_integer()*, *is_float()*, …
- Used when it is necessary to know the type of variable

```php
<?php
  $a=3;
  echo is_integer($a), "#",is_float($a), "\n";
  // stampa 1# con 1 che indica il valore di verità TRUE

 echo gettype($b), "\n"; // stampa integer
 $b = 1.2;
 echo is_integer($b), "#",is_float($b), "#",is_numeric($b), "\n"; // stampa #1#1
 echo gettype($b), "\n"; // stampa double

 $c = "1.2";
 echo is_integer($c), "#", is_float($c), "#", is_numeric($c), "\n"; // stampa ##1
 echo gettype($c), "\n"; // stampa string
>
```

The PHP language syntax

III. Functions

- The PHP language supports *functions,* i.e. the possibility of clustering a number of instructions into independent modules, naming it and passing data to it through function arguments

- A function may be user-defined using the following syntax:

```
<?php
  function foo($arg_1, $arg_2, /* ..., */ $arg_n)
  {
    echo "Example function.\n";
    return $retval;
  }
>
```

- E.g.:

```
<?php
  function sqr($x) {
   return $x*$x;
 }
>
```

The PHP language syntax

III. Functions

- Arguments are passed to functions by value, i.e. changes within the function does not affect externally the value of the passed variable
- Variables defined inside a function have local scope
- Functions may specify default arguments:

```php
<?php
  function my_function($arg1, $arg2="default") {
   //codice
   return ($result);
  }
 $ris1 = my_function("first", "second");// not using default
 $ris2 = my_function("first");// using default
>
```

The PHP language syntax

IV. Global and superglobal variables

- Variables defined externally can only be accessed whithin a function if they are declared *global* inside the function:

```php
<?php
 $a=1;
 function my_function() {
  global $a; // this is necessary to access the external variable
  echo $a;
 }

 my_function();
?>
```

- The *superglobal* variables are special variables that can be accessed also whithin functions without having to declare them as *global.*

The PHP language syntax

IV. Global and superglobal variables

- *Superglobals* are built-in variables that are always available in all scopes
- Superglobals are used to access data from the server running the scripts or from external HTTP requests.
- Several predefined variables in PHP are "superglobals“: there is no need to declare them as *global* to access them within functions.
- Examples of superglobal PHP variables are:

  $_SERVER (Server and execution environment information)
  $_GET (variables passed to the current script via the URL parameters)
  $_POST (variables passed to the current script via the POST method)
  $_COOKIE
  $_SESSION
  …

The PHP language syntax

IV. Constant variables

- Defined through the instruction *define(),* or through the operator *const*
- Names of constants do not require the “$” prefix
- Names of variables and constants are case sensitive, and by convention name of constants are always upperase
- Scope of constants is global

Example 1:

```php
<?php
 define(CONST, "A constant value");
 echo CONST; // prints "A constant value"
 echo const; // prints "const" and generates a warning
 //names of variables and constants are case sensitive
?>
```

Example 2 (valid also inside class definition):

```php
const MIN_VALUE = 0.0;

const MAX_VALUE = 1.0;
```

The PHP language syntax

Program flow control

The PHP language syntax

V. Program flow control

- Selection (conditional statements): ***if***

```php
<?php
if ($a > $b)
   echo "a is bigger than b";
?>
```

- Multi instruction ***if*** statements

```php
<?php
if ($a > $b) {
   echo "a is bigger than b";
   //multi instruction block wants curly brackets!
   $b=$a;
}
?>
```

- Nested ***if*** statements

```php
<?php
if ($a > $b){
  if($b > 100)
   echo "a is bigger than 100";
  echo "a is bigger than b";
}
?>
```

The PHP language syntax

V. Program flow control

- Selection (conditional statements): ***if / else***

```php
<?php
 if ($a > $b) {
  echo "a is greater than b";
 } else {
  echo "a is NOT greater than b";
 }
?>
```

- ***elseif :*** combination of ***if*** and ***else***

```php
<?php
  if ($a > $b) {
    echo "a is bigger than b";
  } elseif ($a == $b) {
      echo "a is equal to b";
  } else {
      echo "a is smaller than b";
  }
?>
```

The PHP language syntax

V. Program flow control

- Selection (conditional statements): ***switch / case***

```php
<?php
 ...
 $i=some_function();
 switch ($i) {
  case 0:
      echo "i equals 0";
      break;
  case 1:
      echo "i equals 1";
      break;
  case 2:
      echo "i equals 2";
      break;
  default:
      echo "i is not equal to 0, 1 or 2";
}
?>
```

- When a case statement matches the value of the switch expression PHP begin to execute the statements (continues until the end of the switch block, or until a break statement.

The PHP language syntax

V. Program flow control

- Iteration: pre-evaluation loops (***while***)

```php
<?php
  $i = 1;
  while ($i <= 10) {
    echo $i++;  /* the printed value would be
                   Si before the increment
                   (post-increment) */
  }
?>
```

- Iteration: post-evaluation loops (***do/while***)

```php
<?php
  $i = 0;
  do {
    echo $i;
  }
  while ($i > 0);
?>
```

The PHP language syntax

V. Program flow control

- Iteration: initialize, evaluate and increment loops (***for***)

- Synthax: `for(expr1; expr2; expr3)`
  `statement;`

1. (*expr1*) is evaluated (executed) once unconditionally at the beginning of the loop
2. In the beginning of each iteration, *expr2* is evaluated. If **true**, the nested statement(s) are executed. If **false**, the execution of the loop ends.
3. At the end of each iteration, *expr3* is evaluated.

Note: *expr2* being empty means the loop should be run indefinitely

```php
<?php
 /* example 1 */
 for ($i = 1; $i <= 10; $i++) {
   echo $i;
 }

 /* example 2 */
 for ($i = 1; ; $i++) {
  if ($i > 10) {
      break;
  }
  echo $i;
 }

 /* example 3 */
 $i = 1;
 for (; ; ) {
  if ($i > 10) {
      break;
  }
  echo $i;
  $i++;
 }
?>
```

The PHP language syntax

V. Program flow control

- Iteration over arrays: ***foreach***

- Synthax:

```
1.  foreach (array_expression as $value)
            statement
2.  foreach (array_expression as $key => $value)
            statement
```

```php
<?php
 /* foreach example 1: value only */

 $a = array(1, 2, 3, 17);
 foreach ($a as $v) {
   echo "Current value of \$a: $v.\n";
 }
?>
```

```php
<?php
 /* foreach example 2: key and value */
 $a = array(
  "one" => 1,
  "two" => 2,
  "three" => 3,
  "seventeen" => 17
 );
 foreach ($a as $k => $v) {
  echo "\$a[$k] => $v.\n";
 }
?>
```

The PHP language syntax

V. Program flow control

- Control of loops: ***break*** and ***continue***

```php
<?php
 $arr = array('one', 'two', 'three', 'four', 'stop', 'five');
 for ($i=0;$i<count($arr),$i++) {
   if ($arr[$i] == 'stop') {
       break;
   }
   echo "$arr[$i]<br />\n";
 }
?>
```



output: “one two three four”

```php
<?php
 $arr = array('one', 'two', 'three', 'four', 'almost_five', 'five');
 for ($i=0;$i<count($arr),$i++) {
   if ($arr[$i] == 'almost_five') {
       continue;
   }
   echo "$arr[$i]<br />\n";
 }
?>
```



output: “one two three four five”

V. Program flow control

- Terminate script: ***exit()*** or ***die()***

The exit($message) and die($message) functions print a message and terminate the current script.

```php
<?php
  $data = file_get_contents( "file.txt" );
  if( $data === false ) {
      die( "Failure." ); // I can't continue. Die.
  }
  do_something_important( $data );
?>
```